Por que Python?



1989: Una solución a la complejidad

- Guido van Rossum buscaba automatizar tareas sin la dificultad de **C**.

Código como lenguaje humano

- Diseñado para ser fácil de leer y escribir por cualquier persona.

El "Superpoder": El Intérprete en C

- Creado en **C** para conectar directamente con las librerías de **C/C++**.

El puente hacia la academia

- Científicos no programadores lo adoptaron para el análisis de datos.

La evolución del ecosistema de IA

- **Datos:** NumPy ➔ Pandas ➔ Scikit-Learn.
- **Deep Learning:** TensorFlow (**Gemini**) ➔ PyTorch (**ChatGPT**).

¿Qué vamos a aprender?

Dos herramientas. Una misma idea.

NumPy

La herramienta fundamental para trabajar eficientemente con números, vectores y matrices.

Google Colab

El entorno donde vamos a escribir, ejecutar y documentar nuestro código Python.

Cómo representar datos → cómo operar con ellos → cómo ejecutar nuestro código

El problema: trabajar con muchos números

Imaginemos que tenemos:

1.200 alumnos
50.000 registros
1.000.000 de mediciones
10.000.000 de valores

Y queremos realizar una operación sobre todos ellos.

¿Los procesamos uno por uno?

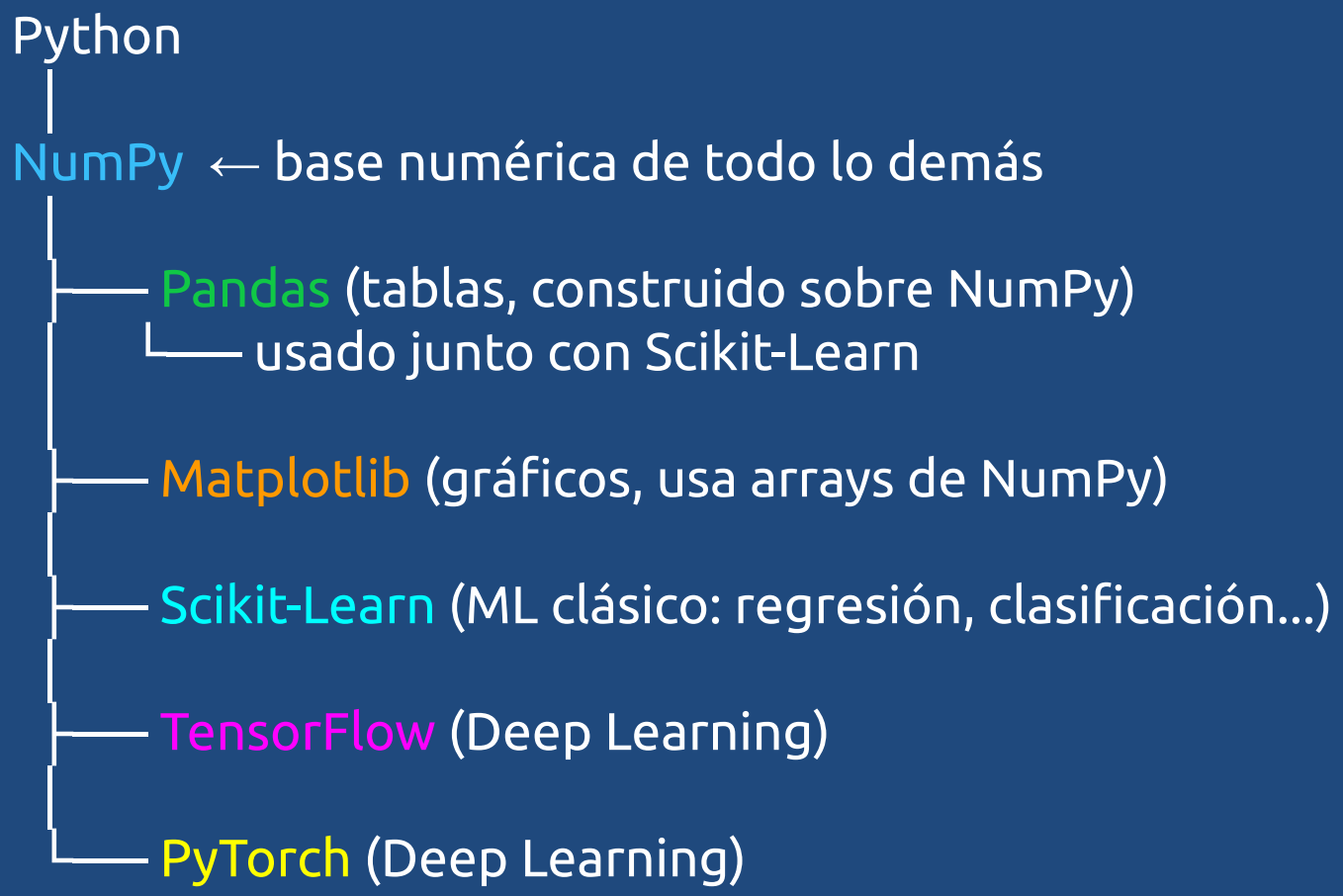
```
for i in range(len(datos)):
    resultado[ i ] = datos[ i ] * 2
```

Python y sus librerías

Python no trabaja solo - Python es un lenguaje de propósito general.

Sobre él se construye un enorme ecosistema de librerías:

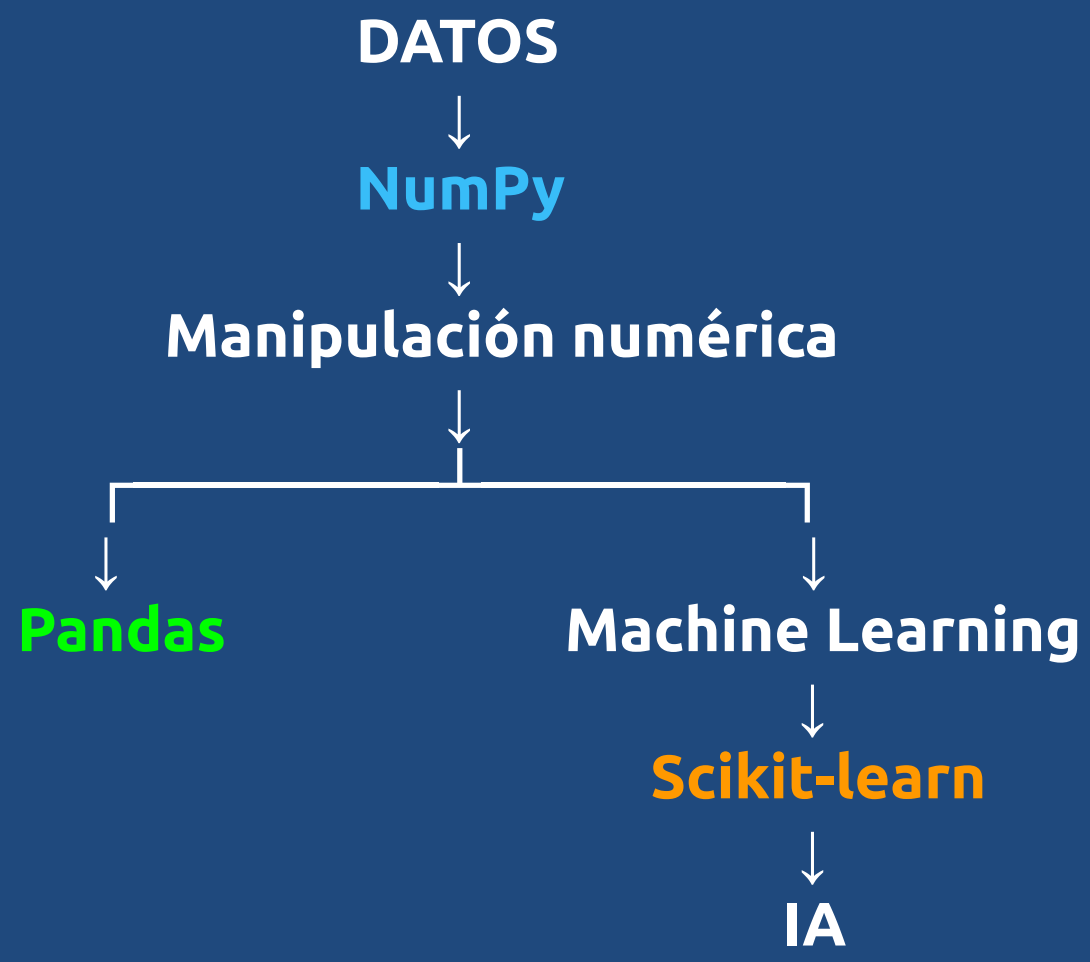
Las librerías permiten reutilizar soluciones ya desarrolladas y optimizadas.



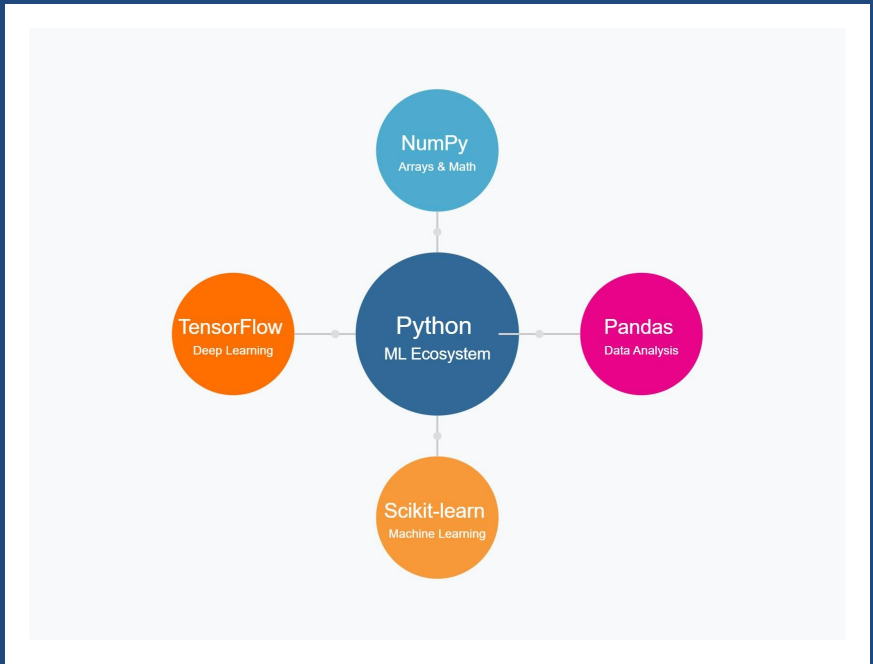
¿Dónde aparece NumPy en IA?

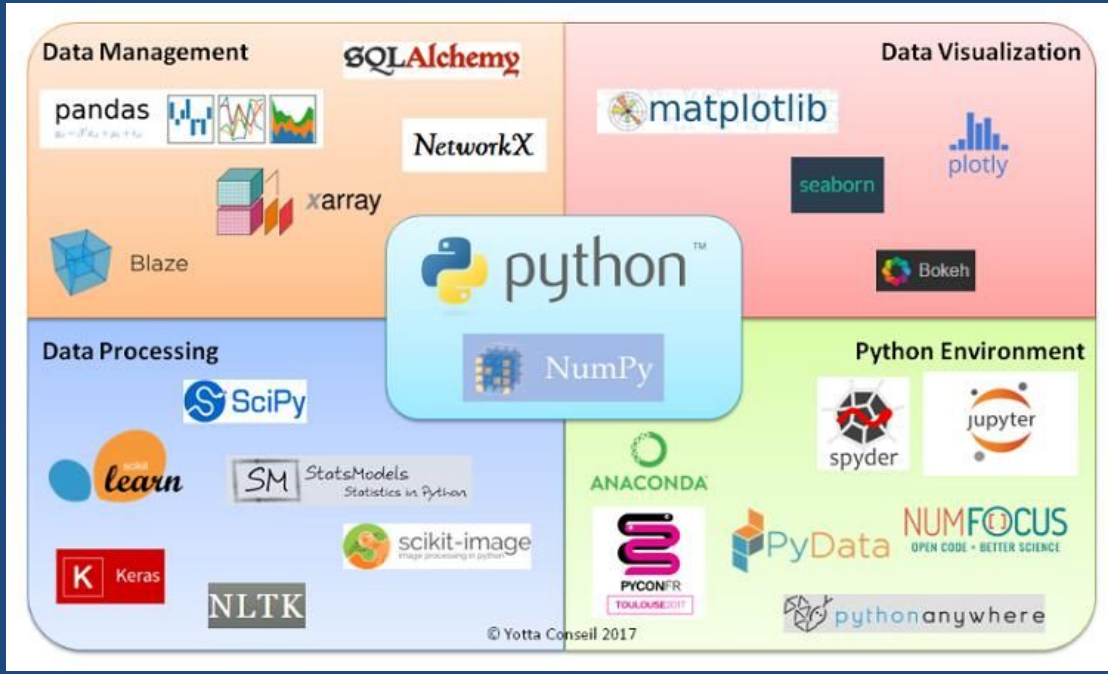
NumPy está mucho más cerca de la IA de lo que parece

NumPy aparece constantemente cuando trabajamos con estructuras numéricas, datos y modelos.



NumPy: los cimientos





Pensemos en NumPy como los cimientos de un edificio.

Sobre sus arrays y operaciones se apoyan herramientas para datos, visualización y aprendizaje automático.

Si entendemos bien el array, tenemos una base fundamental para todo el curso.

Dos formas de guardar datos

Lista de Python:

```
edades = [18, 22, 25, 31]
```

Array de NumPy:

```
edades = np.array([18, 22, 25, 31])
```

A simple vista parecen iguales. Internamente son estructuras diferentes.

Y pensadas para objetivos diferentes.

El problema de sumar listas

a = [1, 2, 3]
b = [4, 5, 6]

Ahora si queremos hacer a + b,

Esperamos [5, 7, 9], pero

Resultado: [1, 2, 3, 4, 5, 6]

En Python, + sobre listas concatena. No suma elemento a elemento.

La solución con un for

```
a = [1, 2, 3]                    # Implementacion en Phyton
b = [4, 5, 6]

resultado = []

for i in range(len( a ) ):
    resultado.append( a[ i ] + b[ i ])
```

FUNCIONA?

La misma operación con NumPy

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b
```

Resultado:

```
array([5, 7, 9])
```

Una sola operación, sin escribir el for.

Vectorización

La idea fundamental

a + b se aplica sobre todos los elementos.

VECTORIZACIÓN: En lugar de recorrer elemento por elemento, expresamos la operación sobre el conjunto completo.

TODOS LOS DATOS

↓

UNA OPERACIÓN

¿Por qué es más rápido?

No es solamente escribir menos código.

NumPy utiliza código optimizado de bajo nivel y estructuras homogéneas de memoria.

Lista de Python:

[objeto]--[objeto]-- [objeto]-- [objeto]

memoria memoria memoria memoria

NumPy:

[1][2][3][4][5][6]

Memoria Contigua

Datos homogéneos + memoria organizada + operaciones optimizadas.

¿Qué es un array?

El ndarray es la estructura central de NumPy

ndarray = N-dimensional array

Puede representar:

- un escalar,
- un vector,
- una matriz o
- estructuras multidimensionales.

Ejemplo:

np.array([10, 20, 30]) - Es un array de una dimensión.

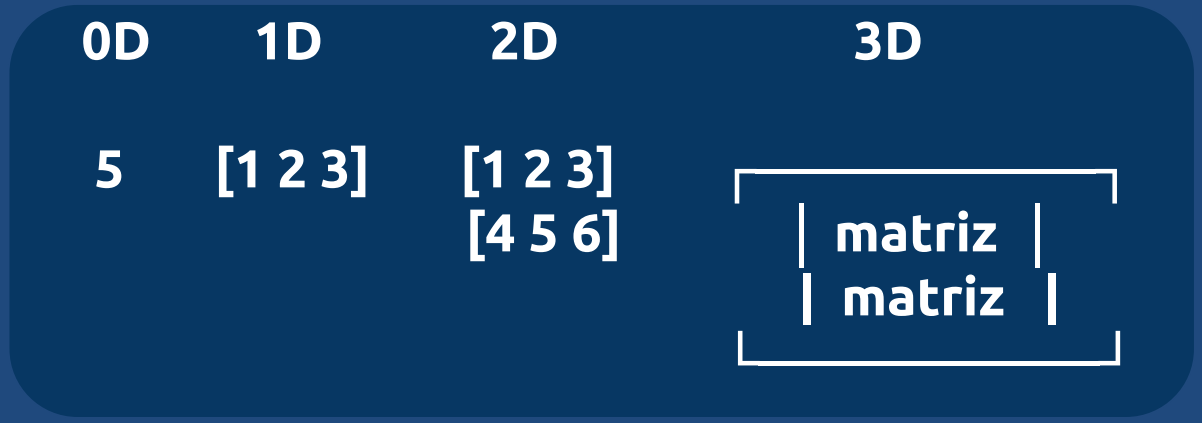
0D, 1D, 2D, 3D...

0D → un valor

1D → un vector

2D → una matriz

3D → una estructura tridimensional



Ejemplos:

1D: vector de edades

2D: tabla de datos

3D: imagen RGB

4D: lote de imágenes para una red neuronal

Lista vs. Array

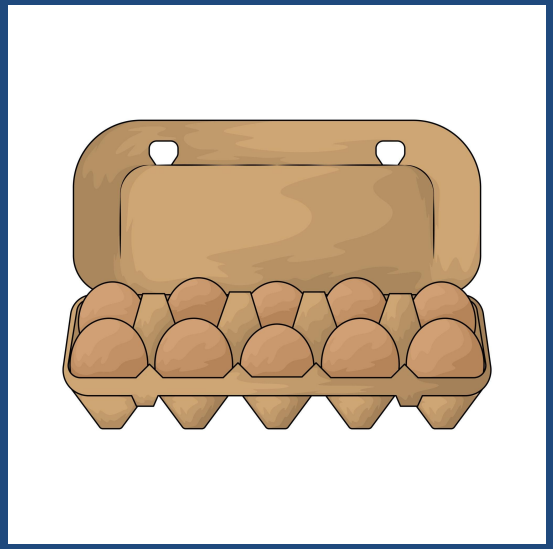
LISTA DE PYTHON

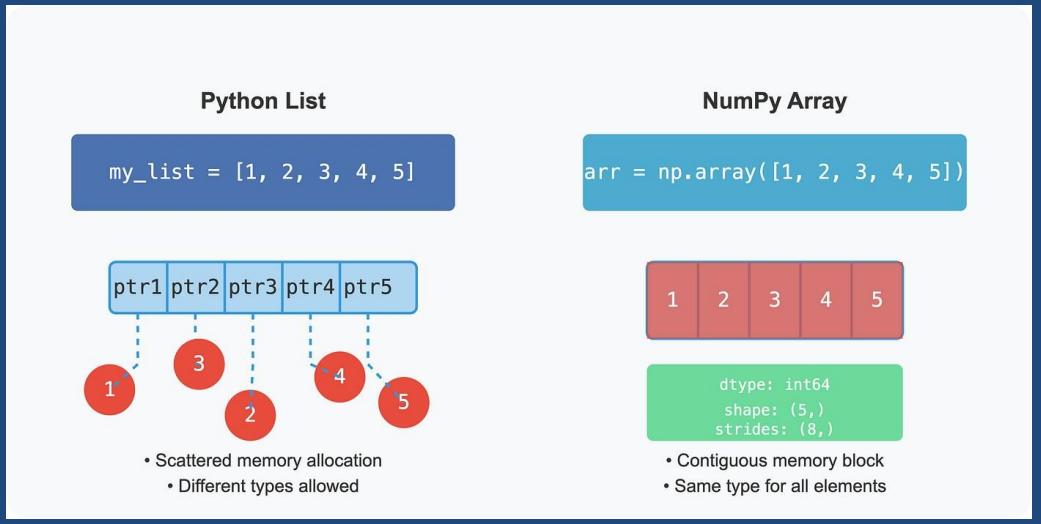
Como un cajón: podemos guardar cosas diferentes y modificarlo libremente.



ARRAY DE NUMPY

Como una bandeja de huevos: espacios organizados, mismo tipo de elemento y estructura definida.





Conociendo nuestro array

Un array no solo guarda números.

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
```

m.dtype → tipo de dato
m.shape → forma
m.ndim → dimensiones
m.size → cantidad de elementos
m.itemsize → bytes por elemento
m.nbytes → memoria total

dtype

¿Qué tipo de datos tenemos?

m.dtype → int64 *(Significa que los elementos son enteros almacenados con ese tipo.)*

np.array([1.0, 2.0, 3.0]).dtype → float64

El tipo importa porque determina cómo se almacenan los datos.

shape y ndim

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
```

m.shape → (2, 3)

2 filas × 3 columnas

m.ndim → 2

Tiene dos dimensiones.

size, itemsize y nbytes

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
```

m.size → 6

m.itemsize → 8 bytes

m.nbytes → 48 bytes

Relación fundamental: nbytes = size × itemsize

48 = 6 × 8

¿Por qué importa el tipo?

Tipo	Memoria típica
int8	1 byte
int16	2 bytes
int32	4 bytes
int64	8 bytes
float16	2 bytes
float32	4 bytes
float64	8 bytes

float64 → 8 bytes por valor

float32 → 4 bytes por valor

En grandes cantidades de datos, float32 puede reducir aproximadamente a la mitad la memoria utilizada.

Esto será especialmente importante al trabajar con datasets y modelos de IA.

¿Cómo creamos arrays?

NumPy ofrece muchas formas

np.array(...)
np.arange(...)
np.linspace(...)
np.zeros(...)
np.ones(...)
np.full(...)
np.eye(...)
np.random...

Cada función responde a una necesidad diferente.

arange vs. linspace

```
np.arange(0, 10, 2)
Resultado :  [0 2 4 6 8]
```

Como range() de Python.

```
np.linspace(0, 1, 5)
Resultado :  [0.00 0.25 0.50 0.75 1.00]
```

Genera una cantidad exacta de valores equiespaciados.

zeros, ones y full

```
np.zeros((2, 3))
```
→ matriz de ceros

```
[0 0 0]
[0 0 0]
```

```
np.ones((2, 3))
```
→ matriz de unos

```
[1 1 1]
[1 1 1]
```

```
np.full((2, 3), 7)
```
→ matriz llena de 7

```
[7 7 7]
[7 7 7]
```

Muy útiles para inicializar estructuras.

Matrices y datos aleatorios

```
np.eye(3)
```

→ matriz identidad

```
[1 0 0]
[0 1 0]
[0 0 1]
```

```
np.random.randint(0, 10, size=5)
```

→ ejemplo: [3 8 1 7 4]

Los datos aleatorios serán fundamentales para experimentar con modelos.

Indexado

```
v = np.array([10, 20, 30, 40, 50])
```

Índice → 0 1 2 3 4
Valor→ 10 20 30 40 50

v[0] → 10
v[2] → 30
v[-1] → 50

Los índices comienzan en 0.

Slicing

v = np.array([10, 20, 30, 40, 50])

Sintaxis general: array[inicio : fin : paso]

ÍNDICE →	**0**	**1**	**2**	**3**	**4**
	↓	↓	↓	↓	↓
VALOR →	**10**	**20**	**30**	**40**	**50**

¿Qué significa cada parte?

inicio → desde dónde comienzo
fin → dónde termino, **sin incluirlo**
paso → cuánto avanzo cada vez

Slicing: Ejemplos

```
v[1:4]

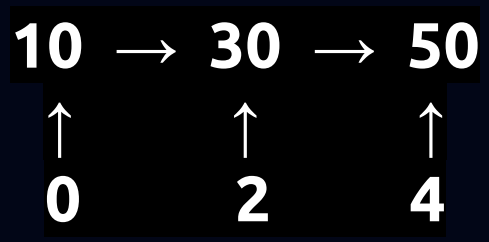
20   30   40
↑           ↑
inicio        fin NO incluido : [20 30 40]

v[:3]    # desde el comienzo hasta 3
v[2:]    # desde 2 hasta el final
v[:]     # todo el array
```

Slicing: Ejemplos

`v[::2]`

`[10 40]`



`v[::-1]`

`[50 40 30 20 10]`

`v[1:4]` → selecciona un rango
`v[:3]` → desde el comienzo
`v[2:]` → hasta el final
`v[:]` → todo
`v[::2]` → salta de 2 en 2
`v[::-1]` → recorre al revés

Arrays 2D

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
```

Accedemos indicando: fila, columna

`m[0, 2]` → 3

`m[1, :]`
→ segunda fila: [4 5 6]

`m[:, 0]`
→ primera columna: [1 4]

Esta forma de acceder a datos será fundamental al trabajar con datasets.

Reshape y datos de IA

¿Por qué necesitamos cambiar la forma?
Un conjunto puede llegar como:

1000 valores

pero un algoritmo puede necesitar:

100 × 10
50 × 20
10 × 10 × 10

reshape() permite reorganizar los datos sin modificar sus valores.

Reshape

```
a = np.arange(12)
```

```
[0 1 2 3 4 5 6 7 8 9 10 11]
```

A otra forma:

```
a.reshape(3, 4)
```

```
[0  1  2  3  ]
[4  5  6  7  ]
[8  9 10 11]
```

Los datos son los mismos; cambió su organización.

Boolean masking

```
edades = np.array([15, 22, 18, 30, 12, 45])

edades >= 18
→ [False True True True False True]

edades[edades >= 18]
→ [22 18 30 45]
```

Filtramos datos sin escribir un for.

Combinar condiciones

Podemos construir filtros más complejos

Mayores o iguales a 18 y menores que 40:

```
edades[(edades >= 18) & (edades < 40)]
```

→ [22 18 30]

& → AND

| → OR

Cada condición debe ir entre paréntesis.

Broadcasting

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

m + 10
```

→ [11 12 13]
  [14 15 16]

NumPy aplica el 10 a todos los elementos.
Esto es broadcasting.

Broadcasting con un vector

```
m = np.array([
                [1, 2, 3],
                [4, 5, 6]
              ])
```

```
fila= np.array([
                 1, 0, 1
               ] )
```

```
m + fila
```

```
[2 2 4]
[5 5 7]
```

El vector se aplica a cada fila sin duplicarlo manualmente.

Funciones de agregación

```
datos = np.array([
                  [1, 2, 3],
                  [4, 5, 6]
                  ])
```

datos.sum() → 21

datos.mean() → 3.5

datos.min() → 1

datos.max() → 6

datos.std() → desviación estándar

¿Qué significa axis?

datos = [1 2 3]
[4 5 6]

sum(axis=0)
→ [5 7 9]
→ resultado por columna

sum(axis=1)
→ [6 15]
→ resultado por fila

Regla práctica:
axis=0 → por columna
axis=1 → por fila

NumPy en una sola idea

NUMPY = DATOS NUMÉRICOS + VELOCIDAD + VECTORIZACIÓN

Un ndarray nos permite trabajar con:

- **Tipos definidos**
- **Memoria organizada**
- **Múltiples dimensiones**
- **Operaciones vectorizadas**
- **Broadcasting**
- **Agregaciones**
- **Filtrado mediante máscaras**

La base para trabajar con datos e IA.

¿Qué es Google Colab?

Nuestro laboratorio de trabajo.

Google Colab permite:

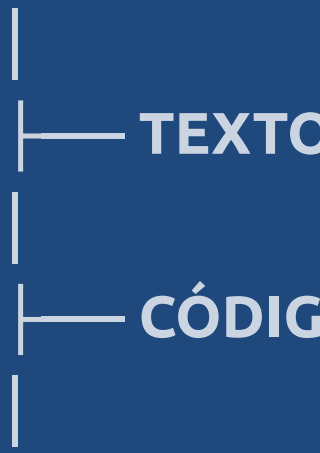
- Escribir Python
- Ejecutarlo
- Ver resultados
- Documentarlo
- Compartirlo

Todo desde el navegador.

El notebook se ejecuta en una máquina virtual de Google.

Notebook + entorno + práctica

GOOGLE COLAB



- TEXTO → explicaciones
- CÓDIGO → Python + NumPy
- RESULTADOS → experimentación

A partir de ahora:

Abrimos nuestro notebook y llevamos todos estos conceptos al código.